

8 AXIS ROBOTIC ARM

An open source, 8-axis ROS2 robotic arm built from scratch to automate complex tasks.

Sushant Thakur (2nd year EE)

The Problem:

Most robotic arms out there are 6DOF at best, and they easily cost upwards of \$5000. For a student or a smaller college, that's just not realistic. The ones that do offer more capability are either proprietary, locked behind expensive licenses, or simply too complex to learn from. There's a huge gap between what's available and what's actually accessible, and that gap is what this project is trying to fill.

Idea:

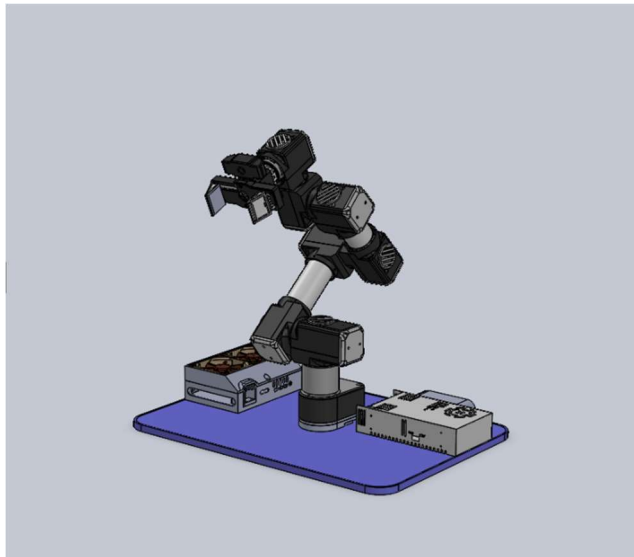


Figure 1: Real-Life Model

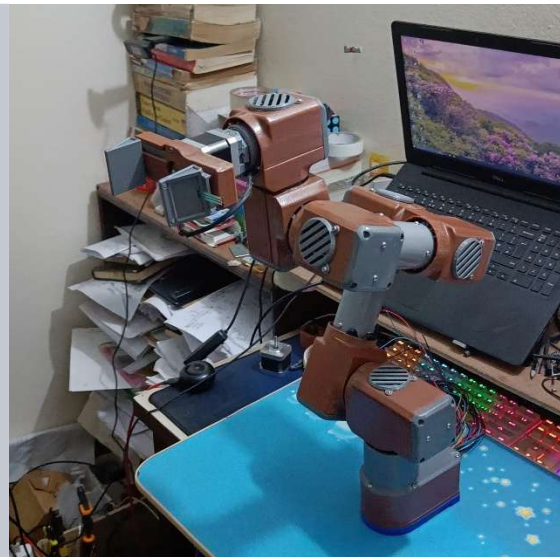


Figure 2: 3D-Model

In daily life and across industries, there are countless repetitive tasks that people want to automate, material handling, packaging, assembly, welding etc. Robotic arms are at the core of all of that, whether it's healthcare, research labs, nuclear facilities, or even space. But the problem is, most robotic arms are expensive and out of reach for students and smaller institutions. So, I built an 8DOF industrial robotic arm from scratch for under \$500 and made it completely open source at <https://github.com/kairobyte/armikochan>, so anyone with a 3D printer and basic electronics can build one, learn from it, and make it better.

How It Works:

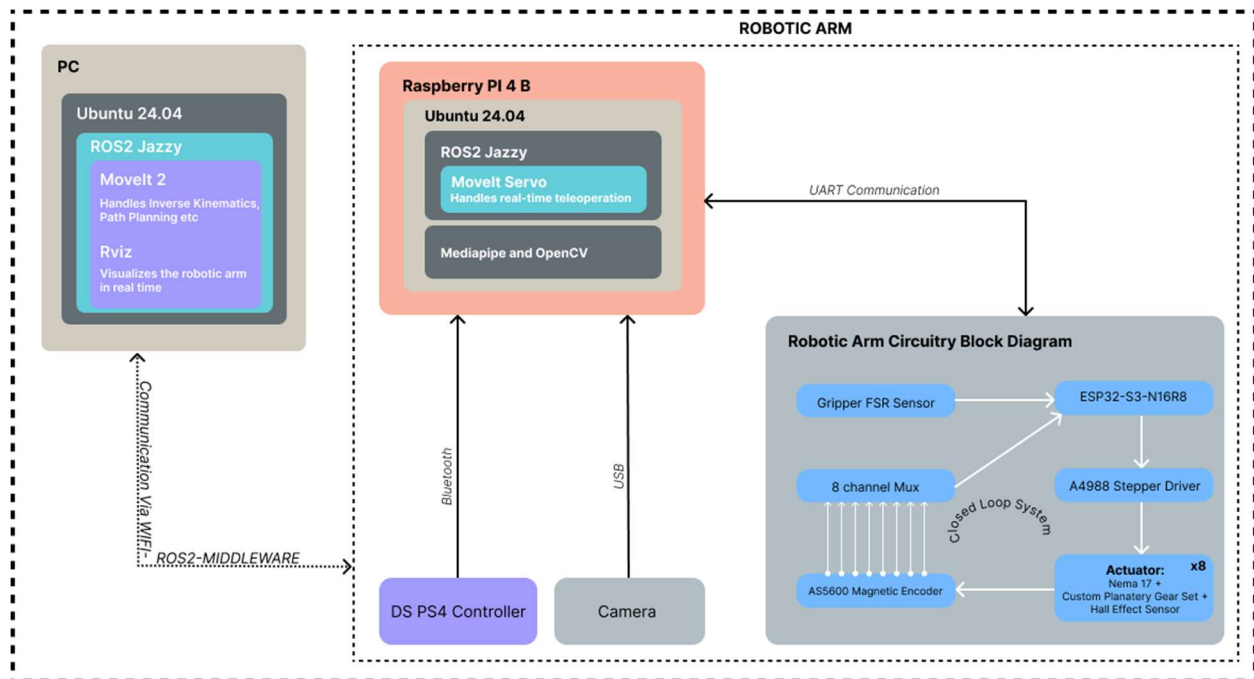


Figure 3: System Layout Diagram

There are two main parts that work together to get things done. Both the PC and the Raspberry Pi run ROS2 Jazzy, which keeps the communication between them really reliable. The PC handles the heavier stuff like Inverse Kinematics, Path Planning, and real-time visualization. The Raspberry Pi takes care of the simpler but time-sensitive tasks like processing camera data.

The robotic arm itself has six main components. The microcontroller is an ESP32-S3-N16R8, and it has enough pins to handle everything I needed to connect, 9 stepper motors, 9 hall effect sensors, and FSR sensors. That's a lot of connections, and most other microcontrollers just don't cut it. For movement, I used Nema 17 stepper motors. The torque isn't enough on their own, so I designed a dual stage planetary gear system that trades some RPM for a much higher torque output.

When a move command comes in through the joystick, the Raspberry Pi picks it up first. MoveIt Servo then breaks that movement into smaller incremental commands, keeping the delay under 40ms from command to execution.

The whole system runs as a closed loop, meaning the arm can correct itself if something pushes it off position. A PID controller checks each joint angle at 40Hz, and if anything is off, it smoothly adjusts back to where it should be.

Result:

In simulation, the arm had an accuracy of about $\pm 4\text{mm}$. In real life though, things like plastic flex and backlash in the actuators brought that down to around $\pm 2\text{cm}$, which is expected, and it's something that can be improved with better materials down the line. The gripper can apply anywhere between 0.4N to 1.5N of force, and the user can adjust that depending on how hard or soft the object is. Adding a thin layer of silicone over the gripper made a noticeable difference too. It increased the friction on the grabbing surface and made the whole thing a lot more reliable. And finally, having those 2 extra degrees of freedom really paid off, the arm was able to avoid obstacles while still doing its task, which is something most budget arms simply can't do.

Challenges:

Designing the planetary gear system in such a tight space, with an outer diameter of just 43mm, was one of the harder parts of this build. The planet gears have really fine teeth, and 3D printing just isn't precise enough at that scale. It works, but it's definitely a weak point.

Another issue was the wiring. The stepper wires and the feedback wires from the AS5600 encoder were running right next to each other, and the electromagnetic interference from the stepper wires was corrupting the feedback signal, making the data unreliable. I fixed this by bumping up the voltage on the feedback line, slowing down the I2C clock speed, binding the wires together, and writing a software debouncing algorithm. Using shielded STP wires would've probably been a cleaner fix, and that's something worth trying in the next version.

Lastly, the motors actually produce more torque than the plastic parts can handle. So, at the end of the day, the plastic is the bottleneck, not the motors.

Next Steps:

There's definitely room to improve the design. For example, the shoulder joint could be made bidirectional, which would help reduce the sag a lot. Switching from plastic to metal parts would also make a big difference, both in accuracy and how reliable the whole system is. The wiring situation could be cleaner too. Right now, the wires pass through a cavity inside the actuator, which limits how far each joint can rotate. Using slip rings instead would remove that limitation and significantly increase the workspace area of the arm.

Team and Tools:

This entire project was done by me, from designing the joints to programming the microcontroller to putting together the electronics. I used SolidWorks for all the 3D design work, and ROS2 for building the control system. For the PCB, I designed everything in KiCad, but in the end I just went with a matrix board and soldered the components by hand.

Now, I'll be the first to admit, building something like this alone is difficult. And when you're doing it all by yourself, your perspective is limited. So not everything is done the most efficient way, and there's honestly a lot of room for improvement.